

Resumen para el Parcial: Conmutación y Tráfico en Redes

Parte 1 — Conmutación de Circuitos y Paquetes

1. ¿Qué es la conmutación?

La conmutación es el mecanismo que interconecta nodos en una red para establecer una vía de comunicación entre un emisor y un receptor, transfiriendo datos de forma eficiente a través de la infraestructura compartida. Los dispositivos (switches, routers, centrales) deciden por dónde reenviar la información examinando solo los encabezados, sin tocar el payload.

Objetivo: que la información llegue al destino con la menor latencia y menor cantidad de pérdidas posibles, sin importar la carga de la red.

1.1 Características de las redes conmutadas

- **Multiplexación y compartición del medio:** varias comunicaciones comparten los mismos enlaces físicos sin mezclarse.
- **Temporalidad y asignación dinámica de recursos:** los recursos (ranuras de tiempo, buffers, etiquetas) se asignan durante la comunicación y se liberan al finalizar.
- **Escalabilidad:** la red crece agregando nodos/enlaces sin rediseñar todo el esquema.
- **Tolerancia a fallos (redes de paquetes):** si un nodo/enlace falla, se pueden recalcular rutas alternativas automáticamente.

2. Tipos de conmutación



Comparación: conmutación de paquetes (recursos compartidos) vs. conmutación de circuitos (canales dedicados).

2.1 Conmutación de circuitos físicos

Se establece un camino dedicado y exclusivo entre los extremos antes de transmitir. Los recursos quedan reservados durante toda la comunicación, aunque haya silencios. Ejemplo típico: la red telefónica tradicional (PSTN).

Fases de la comunicación

- **1. Establecimiento del circuito (señalización):** el emisor solicita conexión; los nodos intermedios reservan recursos y crean un canal fijo; el receptor acepta.
- **2. Transferencia de datos:** los datos fluyen de forma continua a velocidad fija (ancho de banda constante).
- **3. Liberación del circuito:** se liberan los recursos reservados al finalizar.

Ejemplo PSTN: el usuario descuelga → tono de marcado → se marca el número → se transmite por centrales jerárquicas (locales, primarias, secundarias, terciarias) → si ambos están en la misma central, la llamada no sale de ella; si no, se cursa por enlaces troncales y centrales intermedias.

Ventajas vs. Desventajas — Conmutación de circuitos

- ✓ Alta disponibilidad una vez establecido el circuito (como un cable directo, sin lógica de red en los terminales)
- ✓ Transparencia para el usuario e ideal para voz
- ✗ Consume recursos aunque no haya transferencia de datos
- ✗ Ineficiente para tráfico de datos (a ráfagas) y hay retardo en el establecimiento de la llamada

¿Por qué es ineficiente para datos? La voz ocupa el canal de forma casi continua, con pocos silencios, y necesita velocidad constante y retardo predecible: la conmutación de circuitos calza perfecto. El tráfico de datos, en cambio, es a ráfagas (actividad breve + pausas largas), y durante esas pausas el circuito queda reservado pero ocioso. Además, la velocidad fija del circuito no se adapta a dispositivos con capacidades muy distintas. Esto motivó el desarrollo de la conmutación de paquetes.

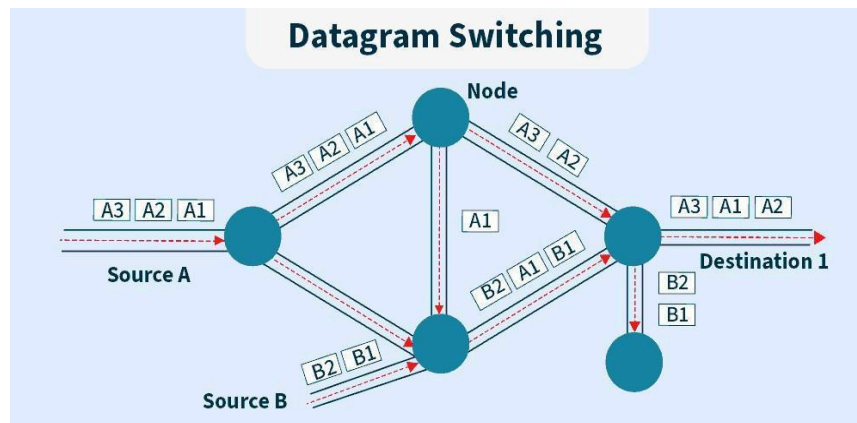
2.2 Conmutación de paquetes (Packet Switching)

La información se divide en bloques llamados paquetes, que viajan del emisor al receptor por una red compartida. A diferencia de la conmutación de circuitos, los recursos no se dedican en exclusiva a una transmisión.

Proceso (3 pasos)

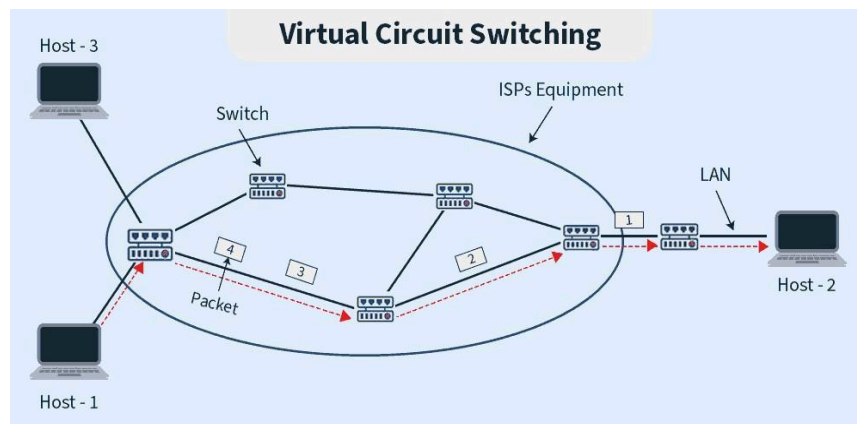
- **1. Segmentación en el origen:** el mensaje se divide en paquetes, cada uno con su propia cabecera (incluye destino).
- **2. Transmisión por la red:** los paquetes pasan por nodos intermedios que los encaminan al destino.
- **3. Reensamblado en el destino:** al llegar todos los paquetes, se reordenan y se reconstruye el mensaje original.

Tipos de conmutación de paquetes



Conmutación por datagramas: cada paquete es independiente y puede seguir una ruta distinta (modelo de Internet, IP).

Cada paquete se trata de forma independiente: lleva la dirección de destino completa y cada nodo lo encamina individualmente (pueden seguir rutas diferentes). Es el modelo usado en Internet (IP).



Conmutación por circuitos virtuales: los paquetes siguen un camino lógico predefinido identificado por VPI.

Antes de enviar datos se establece un camino lógico (circuitos virtuales). Los paquetes llevan un identificador de circuito (VPI - Virtual Path Identifier) y todos siguen la misma ruta predefinida. Se usó históricamente en Frame Relay y ATM; hoy la tecnología dominante es MPLS (Multiprotocol Label Switching), con más flexibilidad, escalabilidad y soporte de QoS.

Tamaño de paquetes y MTU

MTU (Maximum Transmission Unit): tamaño máximo de un paquete. Si un mensaje lo supera, debe fragmentarse. En redes Ethernet (IEEE 802.3) el MTU por defecto es 1500 bytes (en túneles/VPN puede ser menor).

- Paquetes muy grandes → más latencia para el resto del tráfico (acaparan el enlace).
- Mayor probabilidad de congestión (buffers se llenan más rápido).
- Ante un error, se pierde más información y la retransmisión es más costosa.

Por eso se busca el Path MTU óptimo, evitando la fragmentación cuando es posible.

Ventajas vs. Desventajas — Conmutación de paquetes

- ✓ Eficiencia en el uso del ancho de banda (recursos solo cuando hay datos)
- ✓ Mayor tolerancia a fallos (datagramas pueden redirigirse por otras rutas)
- ✓ Flexibilidad (voz, video, datos) y escalabilidad (Internet)
- ✗ Retrasos variables (latencia) por congestión y rutas distintas
- ✗ Complejidad de enrutamiento; desorden en la entrega (hay que reordenar/reensamblar)
- ✗ Mayor carga de procesamiento en routers/switches; posible pérdida de paquetes

Dispositivos involucrados

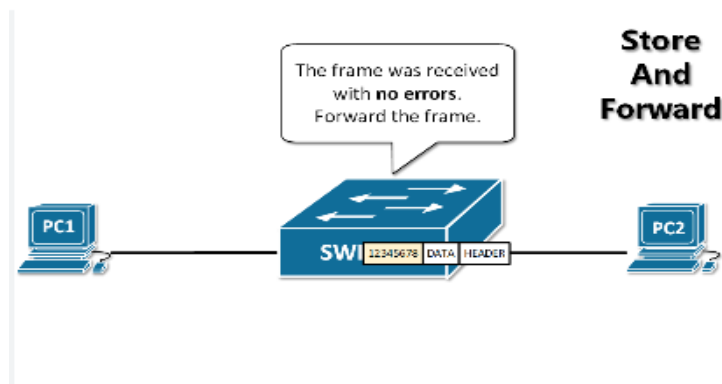
- **Switches (Capa 2):** conmutan tramas según la MAC destino, usando una tabla CAM (MAC ↔ puerto físico). Solo comunican dispositivos de la misma VLAN.
- **Routers (Capa 3):** deciden la ruta de cada paquete IP mediante una tabla de ruteo (redes destino, interfaz de salida, métricas/costos). Se usa el Gateway cuando el destino no pertenece a la propia red IP.

Procesamiento en un router al recibir un paquete:

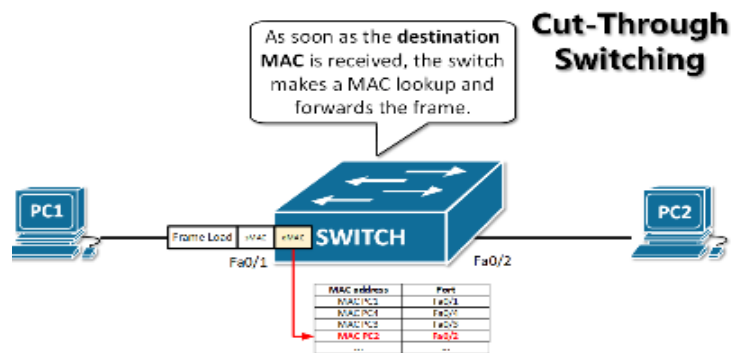
1. Lee la IP de destino del encabezado.
2. Consulta la tabla de ruteo para elegir la interfaz de salida.
3. Si el enlace está libre, encola y transmite el paquete.
4. Si está ocupado, lo almacena temporalmente en un buffer.

El buffering absorbe ráfagas de tráfico, pero agrega retardo variable (jitter); si el buffer se llena, los nuevos paquetes se descartan (p. ej. colas FIFO).

Técnicas de conmutación interna



Store-and-forward: el switch recibe la trama completa, verifica errores (CRC) y luego la reenvía.



Cut-through: el switch reenvía apenas lee la dirección de destino, sin esperar la trama completa.

Campo	Descripción
Store-and-forward	Recibe la trama completa, verifica integridad (CRC) y si es válida la reenvía. Ventaja: elimina tramas corruptas. Desventaja: mayor latencia.
Cut-through	Reenvía apenas lee la dirección destino, sin esperar el resto. Ventaja: baja latencia. Desventaja: puede propagar tramas defectuosas.
Routers	Solo usan store-and-forward: deben procesar el paquete completo (decrementar TTL, fragmentar si supera el MTU, etc.) antes de reenviarlo.

- **Cut-through:** para entornos donde la latencia es crítica (data centers de alto rendimiento, trading financiero, clusters), asumiendo un medio limpio (baja tasa de error).
- **Store-and-forward:** estándar en redes empresariales/campus/acceso; obligatorio si los puertos operan a distinta velocidad.
- **Modo adaptativo:** muchos switches modernos combinan ambos, cambiando de cut-through a store-and-forward si detectan más errores.

Parte 2 — Tráfico Orientado y No Orientado a la Conexión

El tráfico se clasifica según cómo se gestiona la comunicación: orientado a la conexión (TCP) y no orientado a la conexión (UDP). La elección depende de las necesidades de la aplicación (fiabilidad, velocidad, orden) y de la red.

1. Tráfico orientado a la conexión — TCP

Es un acuerdo de estado entre ambos extremos (números de secuencia, confirmaciones, buffers) que permite reconstruir el flujo de datos completo y en orden en el destino. Protocolo de referencia: TCP (capa 4 - Transporte, modelo OSI).

Características generales de TCP

- **Orientado a la conexión:** requiere establecimiento previo (three-way handshake) y cierre controlado.
- **Fiabilidad:** garantiza entrega mediante ACK, retransmisión de paquetes perdidos y detección de errores.
- **Orden:** numera los segmentos para reordenarlos si llegan desordenados.
- **Control de flujo:** evita que un emisor rápido sature a un receptor lento, mediante ventana deslizante.

Cabecera TCP



Cabecera TCP: 160 bits sin opciones (192 con opciones).

TCP agrega una cabecera de al menos 160 bits (192 con opciones) → más latencia de procesamiento. UDP en cambio usa solo 64 bits y 4 campos → menor retardo.

Campo	Descripción
Nro. de secuencia (seq)	Posición del primer byte del segmento dentro del flujo. Sirve para reordenar datos y detectar duplicados.
Confirmación (ack)	Siguiente nro. de secuencia que el receptor espera. Se calcula: seq recibido + bytes de longitud. Ej: seq=100 + 4 bytes → ack=104.
Indicadores (flags)	SYN: inicia la conexión. ACK: valida el campo de confirmación. FIN: solicita cierre ordenado.

Campo	Descripción
Ventana (window)	Bytes que el receptor está dispuesto a aceptar (rwnd). Base del control de flujo. Máximo por defecto: 65.535 bytes (ampliable con Window Scale).

ISN — Número de Secuencia Inicial

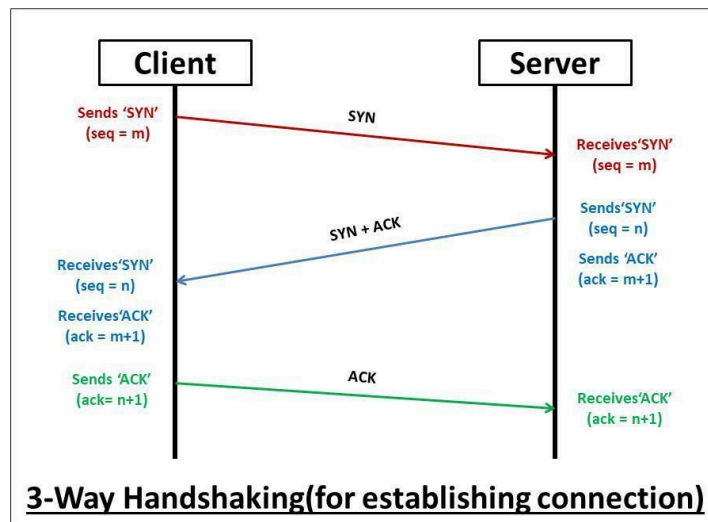
Cada extremo elige aleatoriamente su ISN al iniciar la conexión (no empieza en 1, para evitar confusión con conexiones anteriores). A partir de ahí se cuentan todos los bytes enviados.

Ejemplo SEQ / ACK

Campo	Descripción
Cliente → Servidor: "Hola"	SEQ=1001, ACK=5001. El cliente envía bytes 1001-1004 y espera el byte 5001 del servidor.
Servidor → Cliente: "OK"	SEQ=5001, ACK=1005. El servidor envía bytes 5001-5002. ACK=1005 ("recibí hasta 1004, espero 1005").
Cliente → Servidor (solo ACK)	SEQ=1005, ACK=5003 (5001+2 bytes → "recibí hasta 5002, espero 5003").

Three-way handshake

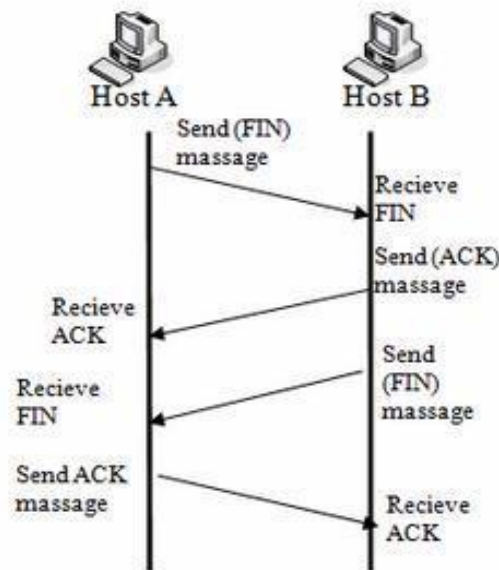
Proceso de 3 pasos (SYN, SYN-ACK, ACK) con el que TCP establece la conexión, sincronizando los ISN de ambos extremos antes de transmitir datos.



3-Way Handshake: SYN → SYN+ACK → ACK.

Cierre de conexión

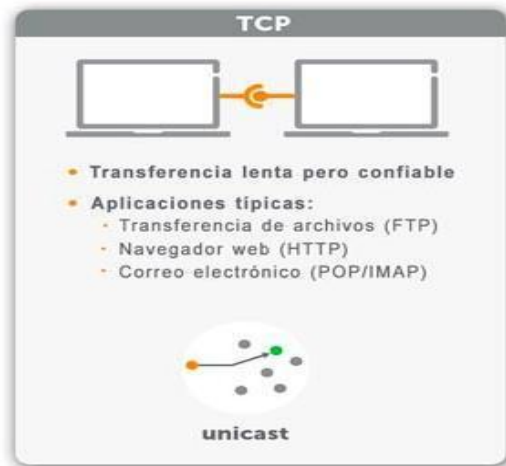
Intercambio de cuatro segmentos (FIN, ACK, FIN, ACK) con el que ambos extremos finalizan la comunicación de forma ordenada, asegurando que todos los datos pendientes se entreguen antes de liberar los recursos.



Cierre de conexión TCP: FIN → ACK → FIN → ACK entre Host A y Host B.

Tipo de tráfico y aplicaciones de capa 7

TCP se usa para tráfico unicast (uno a uno). La sobrecarga de procesar sus cabeceras de control hace difícil implementarlo en multicast o broadcast.



TCP: transferencia lenta pero confiable. Aplicaciones típicas: FTP, HTTP, POP/IMAP. Tráfico unicast.

2. Tráfico no orientado a la conexión — UDP

Los datos se envían directamente, sin establecer conexión previa ni mantener estado entre los extremos. Se prioriza la velocidad y la sencillez sobre la fiabilidad. Protocolo de referencia: UDP (User Datagram Protocol), capa 4 - Transporte.

Características principales de UDP

- **No orientado a la conexión:** no requiere handshake previo; los datagramas se envían directamente.
- **Mejor esfuerzo (best effort):** no hay ACK, retransmisiones ni control de flujo. Si un datagrama se pierde, no se recupera.

- **Sin orden garantizado:** cada datagrama es independiente; pueden llegar desordenados o duplicados.
- **Ligero y rápido:** cabecera fija de solo 8 bytes, sin estado que mantener.
- **Sin control de congestión:** la aplicación decide la tasa de envío.

Cabecera UDP



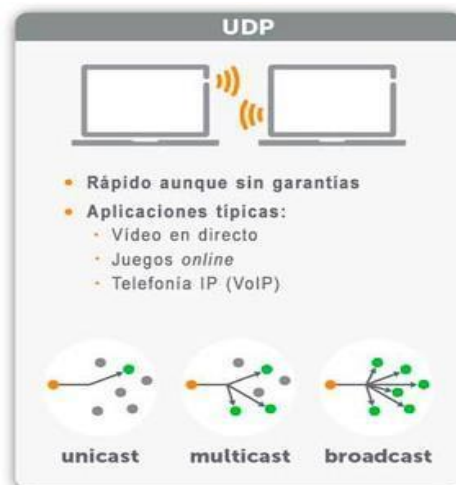
Cabecera UDP: 64 bits en total, 4 campos.

Campo	Descripción
Puerto origen (16 bits)	Identifica el puerto del emisor. Opcional; puede valer 0 si no se requiere respuesta.
Puerto destino (16 bits)	Identifica el puerto del receptor.
Longitud (16 bits)	Tamaño total del datagrama UDP (cabecera + datos).
Checksum (16 bits)	Verifica errores de transmisión. Opcional en IPv4, obligatorio en IPv6.

¿Por qué UDP para broadcast/multicast?

- **TCP es punto a punto:** requiere handshake y estado individual con cada receptor; en broadcast colapsaría la red.
- **UDP es sin conexión:** envía el datagrama a una dirección broadcast o grupo multicast sin importar cuántos receptores haya; cada host decide si acepta o descarta.

Además, en streaming en vivo, VoIP o juegos, perder algún paquete ocasional es aceptable, pero el retraso de TCP (confirmaciones + retransmisiones) arruinaría la experiencia en tiempo real.



UDP: rápido aunque sin garantías. Soporta unicast, multicast y broadcast. Apps: video en directo, juegos online, VoIP.

Aplicaciones típicas de UDP

- Transmisiones en tiempo real: VoIP, videollamadas, streaming en vivo.
- Juegos online (una pérdida ocasional es preferible a un retraso).
- Consultas rápidas: DNS, NTP.
- Protocolos de descubrimiento: DHCP, SSDP.

3. Cuadro comparativo rápido — TCP vs. UDP

TCP	UDP
Orientado a la conexión (handshake)	No orientado a la conexión
Fiable: ACK, retransmisión, control de errores	Best effort: sin ACK ni retransmisión
Garantiza orden (numeración de segmentos)	Sin orden garantizado
Control de flujo (ventana deslizante)	Sin control de flujo ni congestión
Cabecera de 160-192 bits (más latencia)	Cabecera fija de 64 bits (menos latencia)
Tráfico unicast (uno a uno)	Unicast, multicast y broadcast
FTP, HTTP, POP/IMAP	VoIP, streaming, juegos online, DNS, DHCP

Para recordar el día del parcial

- Conmutación de circuitos = canal dedicado fijo (PSTN); ideal para voz, ineficiente para datos a ráfagas.
- Conmutación de paquetes = datagramas (IP, rutas variables) o circuitos virtuales (VPI, ruta fija — Frame Relay/ATM/MPLS).
- MTU típico en Ethernet = 1500 bytes; si se supera, se fragmenta.
- Switch = capa 2, tabla CAM (MAC↔puerto). Router = capa 3, tabla de ruteo, siempre store-and-forward.
- TCP = confiable, ordenado, con conexión, unicast (SYN/SYN-ACK/ACK ; cierre FIN/ACK/FIN/ACK).
- UDP = rápido, sin garantías, sin conexión, soporta broadcast/multicast (DNS, VoIP, streaming, juegos).